Database Management System Lab Solutions

Course Code: SEPBCAP1.7 | Lab Assignments & SQL Queries Solutions

CASE STUDY 1: SELECT CLAUSE & ARITHMETIC OPERATORS

Database: Employee

Task: **Table Schema & Data Insertion**

```
CREATE TABLE Employee (
    Emp_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Hire_Date DATE,
    Address VARCHAR(100),
    City VARCHAR(50)
);

CREATE TABLE EmpSalary (
    Emp_Id INT FOREIGN KEY REFERENCES Employee(Emp_Id),
    Salary DECIMAL(10,2),
    Benefits DECIMAL(10,2),
    Designation VARCHAR(50)
);

INSERT INTO Employee VALUES
(1001, 'Raghav', 'Patel', '2006-05-11', '83 first street', 'Belagavi'),
(1002, 'Mohini', 'Rathi', '2008-02-25', '842 Vine Ave', 'Athani'),
(1012, 'Sham', 'Pujari', '2005-09-12', '33 Elm St.', 'Mysore'),
(1015, 'Kriti', 'Dev', '2006-12-19', '11 Red Road', 'Belagavi'),
(1016, 'Sarath', 'Sharma', '2007-08-22', '440 MG Road', 'New Delhi'),
(1020, 'Monika', 'Gupta', '2008-06-07', '9 Bandra', 'Mumbai');

INSERT INTO EmpSalary VALUES
(1001, 10000, 3000, 'Manager'),
(1002, 8000, 1200, 'Salesman'),
(1012, 20000, 5000, 'Director'),
(1015, 6500, 1300, 'Clerk'),
(1016, 6000, 1000, 'Clerk'),
(1020, 8000, 1200, 'Salesman');
```

Q1. **Display FIRSTNAME, LASTNAME, ADDRESS, and CITY of all employees living in PARIS.**

```
SELECT First_Name, Last_Name, Address, City
FROM Employee
WHERE UPPER(City) = 'PARIS';
```

Q2. **Display the content of the employee table in descending order of FIRSTNAME.**

```
SELECT * FROM Employee
ORDER BY First_Name DESC;
```

Q3. **Select the FIRSTNAME and SALARY of the salesman.**

```
SELECT E.First_Name, S.Salary
FROM Employee E
JOIN EmpSalary S ON E.Emp_Id = S.Emp_Id
WHERE S.Designation = 'Salesman';
```

Q4. Display FIRSTNAME, LASTNAME, and TOTAL SALARY of all employees (TOTAL SALARY = SALARY + BENEFITS).

```
SELECT E.First_Name, E.Last_Name, (S.Salary + S.Benefits) AS TOTAL_SALARY
FROM Employee E
JOIN EmpSalary S ON E.Emp_Id = S.Emp_Id;
```

Q5. List the Names of employees who are more than 1 year old in the organization.

```
SELECT First_Name, Last_Name
FROM Employee
WHERE Hire_Date <= DATEADD(year, -1, GETDATE()); -- (Or DATEDIFF(year, Hire_Date, GETDATE())
> 1)
```

Q6. Count the number of distinct DESIGNATION from EMPSALARY.

```
SELECT COUNT(DISTINCT Designation) AS Distinct_Designations
FROM EmpSalary;
```

Q7. List the employees whose names have exactly 6 characters.

```
SELECT * FROM Employee
WHERE LENGTH(First_Name) = 6; -- (Use LEN() for SQL Server)
```

Q8. Add new column PHONE_NO to EMPLOYEE and update the records.

```
ALTER TABLE Employee ADD PHONE_NO VARCHAR(15);
UPDATE Employee SET PHONE_NO = '9876543210' WHERE Emp_Id = 1001;
```

Q9. List employee names who have joined before 15-Jun-08 and after 16-Jun-07.

```
SELECT First_Name, Last_Name
FROM Employee
WHERE Hire_Date BETWEEN '2007-06-17' AND '2008-06-14';
```

Q10. Generate Salary slip with Name, Salary, Benefits, HRA-50%, DA-30%, PF-12%, Calculate gross. Order descending by gross.

```
SELECT
    E.First_Name, E.Last_Name, S.Salary, S.Benefits,
    (S.Salary * 0.50) AS HRA,
    (S.Salary * 0.30) AS DA,
    (S.Salary * 0.12) AS PF,
    (S.Salary + S.Benefits + (S.Salary * 0.50) + (S.Salary * 0.30) - (S.Salary * 0.12)) AS
Gross_Salary
FROM Employee E
JOIN EmpSalary S ON E.Emp_Id = S.Emp_Id
ORDER BY Gross_Salary DESC;
```

CASE STUDY 2: GROUP BY & HAVING CLAUSE

Database: Bank System

Task: Table Schema & Data Insertion

```
CREATE TABLE Account (Account_No VARCHAR(20), Cust_Name VARCHAR(50), Branch_Id VARCHAR(10));
CREATE TABLE Branch (Branch_Id VARCHAR(10), Branch_Name VARCHAR(50), Branch_City
VARCHAR(50));
CREATE TABLE Depositor (Account_No VARCHAR(20), Branch_Id VARCHAR(10), Balance
DECIMAL(10,2));
CREATE TABLE Loan (Account_No VARCHAR(20), Branch_Id VARCHAR(10), Balance DECIMAL(10,2));
```

```
INSERT INTO Account VALUES
('AE0012856', 'Reena', 'SB002'), ('AE1185698', 'Akhil', 'SB001'),
('AE1203996', 'Daniel', 'SB004'), ('AE1225889', 'Roy', 'SB002'),
('AE8532166', 'Sowparnika', 'SB003'), ('AE8552266', 'Anil', 'SB003'),
('AE1003996', 'Saathwik', 'SB004'), ('AE1100996', 'Swarna', 'SB002');

INSERT INTO Branch VALUES
('SB001', 'Malleshwaram', 'Bangalore'), ('SB002', 'MG Road', 'Bangalroe'),
('SB003', 'MG Road', 'Mysore'), ('SB004', 'Jainagar', 'Mysore');

INSERT INTO Depositor VALUES
('AE0012856', 'SB002', 12000), ('AE1203996', 'SB004', 58900),
('AE8532166', 'SB003', 40000), ('AE1225889', 'SB002', 150000);

INSERT INTO Loan VALUES
('AE1185698', 'SB001', 102000), ('AE8552266', 'SB003', 40000),
('AE1003996', 'SB004', 15000), ('AE1100996', 'SB002', 100000);
```

Q1. **Display the Total Number of accounts present in each branch.**

```
SELECT Branch_Id, COUNT(Account_No) AS Total_Accounts
FROM Account
GROUP BY Branch_Id;
```

Q2. **Display the Total Loan amount in each branch.**

```
SELECT Branch_Id, SUM(Balance) AS Total_Loan_Amount
FROM Loan
GROUP BY Branch_Id;
```

Q3. **Display the Total deposited amount in each branch by descending order.**

```
SELECT Branch_Id, SUM(Balance) AS Total_Deposited
FROM Depositor
GROUP BY Branch_Id
ORDER BY Total_Deposited DESC;
```

Q4. **Display max, min loan amount present in each city.**

```
SELECT B.Branch_City, MAX(L.Balance) AS Max_Loan, MIN(L.Balance) AS Min_Loan
FROM Loan L
JOIN Branch B ON L.Branch_Id = B.Branch_Id
GROUP BY B.Branch_City;
```

Q5. **Display average amount deposited in each branch, each city.**

```
SELECT B.Branch_Id, B.Branch_City, AVG(D.Balance) AS Avg_Deposit
FROM Depositor D
JOIN Branch B ON D.Branch_Id = B.Branch_Id
GROUP BY B.Branch_Id, B.Branch_City;
```

Q6. **Display maximum of loan amount in each branch where the balance is more than 25000.**

```
SELECT Branch_Id, MAX(Balance) AS Max_Loan
FROM Loan
WHERE Balance > 25000
GROUP BY Branch_Id;
```

Q7. **Display Total Number of accounts present in each city.**

```sql
SELECT B.Branch_City, COUNT(A.Account_No) AS Total_Accounts
FROM Account A
JOIN Branch B ON A.Branch_Id = B.Branch_Id
GROUP BY B.Branch_City;
```

Q8. **Display all customer details in ascending order of branch_id.**

```sql
SELECT * FROM Account
ORDER BY Branch_Id ASC;
```

Q9. **Update Balance to 26000 where account_no='AE1003996'.**

```sql
UPDATE Loan
SET Balance = 26000
WHERE Account_No = 'AE1003996';
```

Q10. **Display Customer Names with their branch Name.**

```sql
SELECT A.Cust_Name, B.Branch_Name
FROM Account A
JOIN Branch B ON A.Branch_Id = B.Branch_Id;
```

CASE STUDY 3: LOGICAL & RELATIONAL OPERATORS

Database: Library

Task: **Table Schema & Data Insertion**

```sql
CREATE TABLE Books (
    Book_Id VARCHAR(10), Book_Name VARCHAR(100), Author_Name VARCHAR(100),
    Publishers VARCHAR(100), Price DECIMAL(10,2), Type VARCHAR(50), Quantity INT
);

CREATE TABLE Issued (Book_Id VARCHAR(10), Quantity_Issued INT);

INSERT INTO Books VALUES
('C0001', 'The Klone and I', 'Lata Kappor', 'EPP', 355, 'Novel', 5),
('F0001', 'The Tears', 'William Hopkins', 'First Publ', 650, 'Fiction', 20),
('T0001', 'My First C++', 'Brain & Brooke', 'ERP', 350, 'Text', 10),
('T0002', 'C++ Brainwork''s', 'A.W.Rossaine', 'TDH', 350, 'Text', 15),
('F0002', 'Thunderbolts', 'Ana Roberts', 'First Publ.', 750, 'Fiction', 50);

INSERT INTO Issued VALUES
('T0001', 4), ('C0001', 5), ('F0001', 2), ('T0002', 5), ('F0002', 8);
```

Q1. **To show Book name, Author name and price of books of First Publ. publisher.**

```sql
SELECT Book_Name, Author_Name, Price
FROM Books
WHERE Publishers LIKE 'First Publ%';
```

Q2. **Display Book id, Book name and publisher of books having quantity more than 8 and price less than 500.**

```sql
SELECT Book_Id, Book_Name, Publishers
FROM Books
WHERE Quantity > 8 AND Price < 500;
```

Q3. Select Book id, book name, author name of books which is published by other than ERP publishers and price between 300 to 700.

```
SELECT Book_Id, Book_Name, Author_Name
FROM Books
WHERE Publishers != 'ERP' AND Price BETWEEN 300 AND 700;
```

Q4. Generate a Bill with Book_id, Book_name, Publisher, Price, Quantity, 4% of VAT "Total".

```
SELECT Book_Id, Book_Name, Publishers, Price, Quantity,
       (Price * Quantity) AS Subtotal,
       (Price * Quantity * 0.04) AS VAT_4_Percent,
       (Price * Quantity * 1.04) AS Total_Bill
FROM Books;
```

Q5. Display book details with book id's C0001, F0001, T0002, F0002 (Hint: use IN operator).

```
SELECT * FROM Books
WHERE Book_Id IN ('C0001', 'F0001', 'T0002', 'F0002');
```

Q6. Display Book list other than type Novel and Fiction.

```
SELECT * FROM Books
WHERE Type NOT IN ('Novel', 'Fiction');
```

Q7. Display book details with the author name starting with letter "A".

```
SELECT * FROM Books
WHERE Author_Name LIKE 'A%';
```

Q8. Display book details with the author name starting with letter "T" and ending with "S".

```
SELECT * FROM Books
WHERE Author_Name LIKE 'T%s';
```

Q9. Select Book_Id, Book_Name, Author_Name, Quantity Issued where Books.Books_Id = Issued.Book_Id.

```
SELECT B.Book_Id, B.Book_Name, B.Author_Name, I.Quantity_Issued
FROM Books B
JOIN Issued I ON B.Book_Id = I.Book_Id;
```

Q10. List the book_name, Author_name, Price. In ascending order of Book_name and then in descending order of price.

```
SELECT Book_Name, Author_Name, Price
FROM Books
ORDER BY Book_Name ASC, Price DESC;
```

CASE STUDY 4: SET OPERATIONS & SUBQUERIES

Database: Subject

Task: Table Schema & Data Insertion

```
CREATE TABLE Physics (Regno VARCHAR(20), Name VARCHAR(50), Year VARCHAR(20), Combination
VARCHAR(20));
CREATE TABLE Computer_Science (Regno VARCHAR(20), Name VARCHAR(50), Year VARCHAR(20),
Combination VARCHAR(20));

INSERT INTO Physics VALUES
('AJ00325', 'Ashwin', 'First', 'PCM'),
```

```
('AJ00225', 'Swaroop', 'Second', 'PMCs'),
('AJ00385', 'Sarika', 'Third', 'PME'),
('AJ00388', 'Rupa', 'First', 'PMCs');

INSERT INTO Computer_Science VALUES
('AJ00225', 'Swaroop', 'Second', 'PMCs'),
('AJ00296', 'Tejas', 'Second', 'BCA'),
('AJ00112', 'Geeta', 'First', 'BCA'),
('AJ00388', 'Rupa', 'First', 'PMCs');
```

Q1. **Select all students from Physics and Computer Science (Union / All combined).**

```
SELECT Regno, Name, Year, Combination FROM Physics
UNION
SELECT Regno, Name, Year, Combination FROM Computer_Science;
```

Q2. **Select students common in Physics and Computer Science (Intersection).**

```
SELECT P.Regno, P.Name, P.Year, P.Combination
FROM Physics P
INNER JOIN Computer_Science C ON P.Regno = C.Regno;
-- Alternative Standard SQL:
-- SELECT * FROM Physics INTERSECT SELECT * FROM Computer_Science;
```

Q3. **Display all student details those are studying in the second year.**

```
SELECT * FROM Physics WHERE Year = 'Second'
UNION
SELECT * FROM Computer_Science WHERE Year = 'Second';
```

Q4. **Display students who are studying both Physics and Computer Science in the second year.**

```
SELECT P.* FROM Physics P
JOIN Computer_Science C ON P.Regno = C.Regno
WHERE P.Year = 'Second';
```

Q5. **Display the students studying only physics.**

```
SELECT * FROM Physics
WHERE Regno NOT IN (SELECT Regno FROM Computer_Science);
-- Alternative: SELECT * FROM Physics EXCEPT SELECT * FROM Computer_Science;
```

Q6. **Display the students studying only Computer Science.**

```
SELECT * FROM Computer_Science
WHERE Regno NOT IN (SELECT Regno FROM Physics);
```

Q7. **Select all students having PMCs combination.**

```
SELECT * FROM Physics WHERE Combination = 'PMCs'
UNION
SELECT * FROM Computer_Science WHERE Combination = 'PMCs';
```

Q8. **Select all students having a BCA combination.**

```
SELECT * FROM Computer_Science WHERE Combination = 'BCA';
```

Q9. **Select all students studying in the Third year.**

```
SELECT * FROM Physics WHERE Year = 'Third'
UNION
SELECT * FROM Computer_Science WHERE Year = 'Third';
```

Q10. Rename table Computer Science to CS.

```
EXEC sp_rename 'Computer_Science', 'CS';
-- Or standard SQL: ALTER TABLE Computer_Science RENAME TO CS;
```